

ESTUDI ESTADÍSTIC COMPARATIU DELS ALGORISMES BFS I DFS EN FUNCIÓ DE LA PROFUNDITAT DELS NODES



Autors: Hugo Pelayo, Sergio Borque i Daniel Hergueta
Grup: 42

ÍNDEX

INTRODUCCIÓ	2
OBJECTIUS	3
VARIABLES	4
Variables independents	4
Variables dependents	5
Naturalesa i tractament de les dades	5
METODOLOGIA	6
Generació automàtica de grafs	6
Execució de les proves i recollida de resultats	7
Control experimental i garantia de validesa	7
ANÀLISI ESTADÍSTIC	9
Comparació de poblacions	10
Previsió	11
Més comparacions	12
CONCLUSIÓ	

INTRODUCCIÓ

En l'àmbit de la informàtica, els algorismes de cerca en grafs constitueixen un dels fonaments més importants per a la resolució de problemes complexos. Des de la planificació de rutes i la navegació per xarxes fins a la intel·ligència artificial, la capacitat de recórrer un graf de manera eficient és essencial. En aquest context, dos dels algorismes més coneguts i estudiats són la cerca en amplada (*Breadth-First Search*, BFS) i la cerca en profunditat (*Depth-First Search*, DFS).

Malgrat la seva aparent simplicitat, aquests algorismes presenten diferències conceptuals i pràctiques notables. Tots dos permeten explorar un conjunt de nodes i arestes amb l'objectiu de trobar un element concret o determinar l'estructura d'un graf, però la seva manera d'afrontar aquesta exploració és oposada.

La motivació d'aquest treball neix de la necessitat de quantificar i analitzar empíricament aquestes diferències amb la fi d'entendre en quines condicions cadascun d'ells pot ser més eficient o més adequat. Si bé la teoria ens proporciona una visió general sabem que tant BFS com DFS tenen una complexitat temporal de $O(V + E)$ on V és el nombre de nodes i E el d'arestes la seva eficiència real pot variar significativament segons les condicions experimentals. Factors com la densitat del graf, el nombre de connexions per node, la profunditat mitjana dels camins o la estructura de connexió poden afectar el temps d'execució de manera diferent per a cada algorisme, o, fins i tot, la mateixa implementació de tots dos algorismes o la del graf.

OBJECTIUS

L'objectiu principal és comparar el rendiment temporal de BFS i DFS sobre grafs dirigits de densitats i mides diverses i saber quin dels dos és més ràpid. A més, s'abordaran altres objectius secundaris com estudiar la relació entre la densitat i el temps d'execució o el temps requerit per fer la cerca respecte la distància del camí recorregut.

Aquests objectius s'han definit en termes quantificables i observables, fent servir variables numèriques concretes (temps en mil·lisegons de recorregut, nombre de nodes, nombre d'arestes, densitat i longitud del camí, etc.) per tal que l'estudi s'enfoqui en resultats que es poden analitzar estadísticament i representar gràficament.

Per assolir aquest propòsit, s'han establert els següents objectius específics:

1. **Comparar el rendiment temporal de BFS i DFS sobre conjunts de grafs dirigits i determinar si hi ha diferència significativa.** Aquesta mesura es fa a partir del temps total d'execució requerit per completar una cerca entre un node inicial i un node final seleccionats aleatòriament.
2. **Comparar el temps d'execució i la longitud del camí recorregut,** per determinar si existeix relació entre la distància real trobada per cadascun dels algorismes i el cost temporal associat a la cerca, especialment en grafs de gran mida o densitat elevada.

Les mesures es registraran en el fitxer **test_report.csv** per el seu anàlisi posterior.

Hipòtesi

Com a hipòtesi del nostre projecte i per donar suport a l'objectiu, deduïm que els dos algorismes, concretament per aquesta tasca, no haurien de diferir massa, tot i que potser l'algorisme DFS és lleugerament més eficient en cerques amb més profunditat.

Les bases d'aquest pensament s'expliquen perquè, tot i que el BFS està especialitzat en trobar el camí més curt (degut a que l'algorisme retorna quan troba un camí i sempre és el més curt), la poda que s'aplica al DFS provocaria que, a llarg termini i per a un graf dens, la majoria de camins no siguin considerats per a l'avaluació.

VARIABLES

Per tal de garantir que els objectius del treball siguin mesurables, realistes i verificables, s'han definit un conjunt de variables quantitatives que permeten descriure tant les característiques estructurals dels grafs com el comportament dels algorismes aplicats sobre ells. Molt important el fet que les variables que nosaltres podem controlar els seus valors han estat generat dins un rang de forma aleatòria amb una distribució uniforme fent servir servir eines de la llibreria standard de C++, això simplement per fer l'experiment més interessant.

Totes les dades es recullen automàticament durant l'execució del programa de proves (**tester_app**) i s'emmagatzemen en un fitxer CSV per a la seva anàlisi posterior.

Variables independents

Les variables independents (que també es diuen decisions) són aquelles que es defineixen o controlen directament durant la generació dels grafs i que poden influir en el rendiment dels algorismes. El següent conjunt de variables descriu l'escenari estructural sobre el qual operen els algorismes BFS i DFS:

- Nombre de nodes (Nodes): representa la mida del graf (el total de vèrtexs) i es genera de manera aleatòria dins d'un interval prefixat. Determina l'escala del problema i influeix directament en la complexitat temporal de les cerques.
- Nombre d'arestes (Edges): indica el nombre total de connexions dirigides entre nodes. Juntament amb el nombre de nodes, defineix la densitat del graf i la seva complexitat estructural.
- Densitat (Density): es calcula com $D = E / (V * (V - 1))$, on E és el nombre d'arestes i V el nombre de nodes. Aquesta variable mesura el grau de connectivitat global del graf i és clau per entendre el comportament dels algorismes.
- Nodes d'inici i de destí (Start Node, End Node): seleccionats de forma aleatòria per a cada prova, defineixen el punt de partida i el punt final de la cerca. Tot i que són aleatoris, la seva variació introdueix diversitat en les situacions de cerca.

Variables dependents

Les variables dependents (també anomenades respostes) són aquelles que resulten de l'execució dels algorismes i que permeten quantificar-ne el comportament i l'eficiència. Les següents variables permeten establir una comparació directa entre els dos algorismes, tant des del punt de vista temporal com estructural:

- Temps d'execució de BFS (BFS Time (ms)): mesura el temps total, en mil·lisegons, que l'algorisme BFS triga a completar la cerca des del node inicial fins al node final.
- Temps d'execució de DFS (DFS Time (ms)): paral·lel a la variable anterior. Permet comparar directament el cost temporal de DFS amb el de BFS.
- Longitud del camí òptim de BFS i DFS (Path Length): indica el nombre d'arestes que componen el camí trobat entre els dos nodes seleccionats. En grafs no ponderats, aquest valor correspon al camí més curt possible.

Totes les variables són discretes o contínues, la qual cosa permet aplicar tècniques estadístiques clàssiques (mitjanes, desviacions, regressions i proves t). Les mesures es prenen amb alta precisió mitjançant el temporitzador **scoped_timer** i s'emmagatzemen en format decimal de doble precisió per millorar la qualitat de l'estudi¹.

¹ El component **scoped_timer** utilitza el patró RAI, de manera que inicia el comptador en crear-se l'objecte i calcula automàticament el temps transcorregut que el flux d'execució surt de del seu àmbit de visibilitat. Això permet mesurar durades sense necessitat de gestionar manualment l'inici o el final del temporitzador. A més a més en qualsevol moment es pot consultar el temps transcorregut des de la creació de l'objecte.

METODOLOGIA

Per tal de facilitar la comparativa experimental entre els algorismes BFS i DFS, en aquest projecte, s'implementarà una infraestructura pròpia de generació i representació de grafs, mitjançant una classe genèrica `gentree<T>` (basada en un contenidor associatiu `unordered_map` que enllaça cada node amb el conjunt dels seus successors immediats) que permet crear, modificar i serialitzar grafs dirigits de diferents mides i densitats.

A partir d'aquesta base, es realitzaran múltiples experiments controlats, comparant el temps d'execució i la longitud dels camins trobats per BFS i DFS en situacions diverses. Les dades obtingudes com les densitats, els temps mitjans i les longituds de camí entre els nodes inici i destí es registraran per al seu tractament estadístic posterior, utilitzant mètodes d'estadística descriptiva.

La implementació adoptada respon a criteris de simplicitat i eficiència en entorns on els valors dels nodes són de tipus bàsics (com enters o caràcters), comptant amb el fet que la inserció i consulta en un `unordered_map` ofereixen un rendiment mitjà proper a $O(1)$. No obstant això, cal remarcar que aquesta elecció no seria òptima si els nodes representessin objectes més complexos amb múltiples atributs o relacions internes, ja que en aquest cas seria preferible utilitzar estructures més robustes i segures (com ara llistes d'adjacència basades en punters, referències o gestors de memòria dinàmica).

La raó d'aquest fet té a veure amb aspectes tècnics del llenguatge de programació que emprem per aquest experiment, C++. La construcció de tipus no bàsics pot ser no trivial quan s'insereixen noves dades en aquest tipus de contenidors.

Recalquem que, a la implementació actual, els nodes es copien dins de les llistes d'adjacència, en lloc d'emmagatzemar referències o identificadors únics. Aquesta simplificació s'ha fet de manera deliberada, amb l'objectiu de mantenir el focus del treball en l'anàlisi del comportament dels algorismes de cerca i no en la gestió de memòria o optimitzacions d'estructura de dades.

Pel que fa a la implementació dels algorismes, la versió de la cerca en profunditat (DFS) s'ha implementat mitjançant una pila iterativa (`stack`) en lloc d'una versió recursiva tradicional. Aquesta decisió respon a motius de robustesa i control de recursos: la recursivitat, tot i ser més elegant des d'un punt de vista conceptual, pot comportar un risc d'esgotament de l'espai d'execució (*stack overflow*) en grafs de gran mida o amb camins molt profunds. En canvi, l'ús explícit d'una pila permet gestionar millor la memòria i mantenir un comportament previsible en tots els casos.

Aquesta estratègia també facilita la mesura dels temps d'execució, ja que elimina la variabilitat introduïda per la gestió interna de la pila del sistema. En conjunt, aquestes decisions busquen un equilibri entre simplicitat i control experimental, assegurant que els resultats obtinguts reflecteixin principalment les diferències inherents als algorismes i no els efectes secundaris d'una implementació massa sofisticada.

En plantejar aquest treball, s'ha valorat acuradament la viabilitat tècnica i temporal del projecte per assegurar que l'objectiu sigui assolible amb els recursos disponibles i les eines vistes a l'assignatura. La comparativa d'eficiència entre els algorismes BFS i DFS s'ha considerat factible perquè es pot implementar i analitzar completament amb coneixements de programació estructurada, gestió de dades i estadística descriptiva i inferencial, competències ja assolides en el marc del curs. A més, la infraestructura de proves s'ha dissenyat de manera automatitzada, fet que permet generar, executar i registrar centenars de proves sense necessitat d'intervenció manual, reduint així l'esforç temporal i garantint la coherència del conjunt de dades.

Naturalesa i tractament de les dades

El procés de recollida de dades s'ha estructurat al voltant de dos programes principals que treballen de manera complementària:

1. Un mòdul generador de grafs, responsable de crear i emmagatzemar estructures dirigides amb propietats aleatòries controlades.
2. Un mòdul de proves que hem anomenat *"tester_app"*, encarregat d'executar els algorismes BFS i DFS sobre cada graf i registrar els resultats en un fitxer de dades.

Aquesta arquitectura modular permet separar clarament la fase de generació de la fase d'experimentació, garantint que els grafs utilitzats siguin consistents i que els temps mesurats reflecteixin exclusivament el cost de les cerques, no la construcció dels grafs.

Generació automàtica de grafs

La generació dels grafs s'ha dut a terme mitjançant l'aplicació **generator_app**, implementada en C++20. Cada graf es construeix a partir de la classe **gentree<T>**, una estructura genèrica basada en un **unordered_map** que associa cada node amb el conjunt dels seus successors. Aquesta representació ofereix accés constant mitjà a les adjacències i resulta adequada per a tipus de dades bàsics. El nombre de nodes de cada graf es tria de forma aleatòria dins d'un interval configurat (per exemple, entre 500 i 1500) i per a cada node es generen un nombre aleatori d'arestes sortints, controlat pel paràmetre **max_adjacencies_per_node**.

La resta de paràmetres rellevants per a l'estudi també es poden configurar amb un fitxer en amb extensió **.toml** disponible en el projecte.

Els valors aleatoris es generen mitjançant el mòdul **random**, que ens genera números aleatoris distribuïts uniformement dins d'un interval. Per cada connexió, es crea una arista dirigida amb el format **(from → to)**. No estan permesos autoenllaços (connexió d'un node amb ell mateix) per limitacions de l'experiment i la unicitat dels nodes en les adjacències està garantida perquè fem servir un **unordered_set**. Finalment, el graf resultant es serialitza a disc en format de text mitjançant el mètode **serialize()**.

Execució de les proves i recollida de resultats

El segon programa, **tester_app**, és l'encarregat de carregar els grafs generats i d'executar les cerques BFS i DFS sobre cadascun d'ells. Aquest mòdul també fa ús de la llibreria externa *Taskflow* per paral·lelitzar les proves i maximitzar l'ús dels recursos del sistema disponibles. Els fitxers de grafs es carreguen mitjançant **tree_loader**, que reconstrueix la representació interna **gentree<int>** a partir dels fitxers **.txt** generats anteriorment. El format de serialització dels grafs és el següent:

```
# Un número representa un node
1
2
3

# Un parell de números representa una aresta dirigida
1 2 //Aresta de 1 a 2
2 3 //Aresta de 2 a 3
```

Els nombres individuals representen nodes i els parells de nombres [X Y] representa una aresta que va de X a Y, ja que els grafs representats per la classe *gentree* són dirigits.

Per cada graf, el sistema selecciona dos nodes, un d'origen i un de destí, de manera aleatòria. A continuació, s'executen els dos algorismes. El temps d'execució de cada cerca s'obté amb la classe **scoped_timer**, que mesura amb alta precisió el temps transcorregut entre l'inici i el final de l'operació. Les dades recollides s'agrupen en una estructura **test_report** per a cada graf, i posteriorment es serialitza a un fitxer CSV únic (un per totes les proves).

El fitxer conté una línia per graf amb el format:

Graph Name, Density, Nodes, Edges, DFS Time (ms), BFS Time (ms), Path Length, Start Node, End Node

Control experimental i garantia de validesa

Per assegurar que els resultats siguin fiables, cada prova s'ha executat sota condicions controlades: minimitzant el nombre de processos ocupant CPU i amb la mateixa configuració de compilació.

Els experiments s'han paral·lelitzat, però no solapats en el temps dins del mateix graf, evitant efectes de dependència. Així mateix, s'ha verificat que el càlcul de densitat coincideixi amb el nombre d'arestes reals i que els nodes d'origen i destí escollits siguin vàlids dins del conjunt de nodes existents.

A continuació, s'adjunta un exemple de visualització d'un CSV generat:

	Graph Name	Density	Nodes	Edges	DFS Time (ms)	BFS Time (ms)	DFS Path Length	BFS Path Length	Start Node	End Node
1	../generator/resources/graph_707.txt	0.007825254743005123	1147	10286	0.734	0.487	219	3	44	1079
2	../generator/resources/graph_2727.txt	0.007473011822602961	1284	10824	0.563	0.179	159	2	792	954
3	../generator/resources/graph_318.txt	0.008226061523288045	1085	9675	0.927	0.864	195	3	1043	14
4	../generator/resources/graph_2607.txt	0.006697496855765913	1341	12035	1.935	1.252	65	3	218	24
5	../generator/resources/graph_3895.txt	0.008370909141870136	1069	9557	1.065	0.426	147	3	538	213
6	../generator/resources/graph_143.txt	0.0067125032513164345	1339	12026	1.965	2.581	58	4	1303	681
7	../generator/resources/graph_764.txt	0.007467333889352238	1200	10744	1.267	2.362	135	5	1	182
8	../generator/resources/graph_285.txt	0.006051944530135963	1483	13301	0.913	1.599	276	4	4	659
9	../generator/resources/graph_3867.txt	0.00647119826140129	1392	12530	2.022	2.41	65	4	22	386
10	../generator/resources/graph_3640.txt	0.008831169333921755	1017	9125	1.627	1.095	27	3	514	126
11	../generator/resources/graph_426.txt	0.006584275939114649	1365	12259	1.913	0.927	76	3	475	1177
12	../generator/resources/graph_2081.txt	0.007045866264809145	1274	11427	0.027	1.604	5	4	149	150
13	../generator/resources/graph_1340.txt	0.007882436151729016	1141	10253	0.599	2.168	179	4	483	572
14	../generator/resources/graph_389.txt	0.007372347797879712	1222	11000	2.225	2.062	6	4	592	337
15	../generator/resources/graph_4914.txt	0.00890098674358853	1008	9035	0.934	0.222	159	2	35	899
16	../generator/resources/graph_2532.txt	0.006943066255314798	1290	11545	0.779	0.054	234	2	644	285
17	../generator/resources/graph_3970.txt	0.006644112684855031	1349	12082	1.176	1.036	221	3	628	1321
18	../generator/resources/graph_4295.txt	0.007616267077098837	1178	10560	2.074	1.095	17	3	624	399
19	../generator/resources/graph_1481.txt	0.006109973989891526	1472	13230	2.323	1.696	46	4	574	884
20	../generator/resources/graph_225.txt	0.00743516167433923	1206	10805	1.527	1.26	88	3	24	1059
21	../generator/resources/graph_2733.txt	0.006332534715966607	1418	12724	1.805	1.621	103	3	223	53
22	../generator/resources/graph_2309.txt	0.00671861213996838	1337	12001	1.97	2.424	55	4	163	790
23	../generator/resources/graph_941.txt	0.006349199948383766	1409	12596	1.642	0.408	149	3	523	104

ANÀLISI ESTADÍSTIC

Abans de dur a terme la inferència estadística per comparar el rendiment dels algorismes BFS i DFS, farem una validació prèvia de les principals premisses necessàries per garantir la validesa dels tests aplicats.

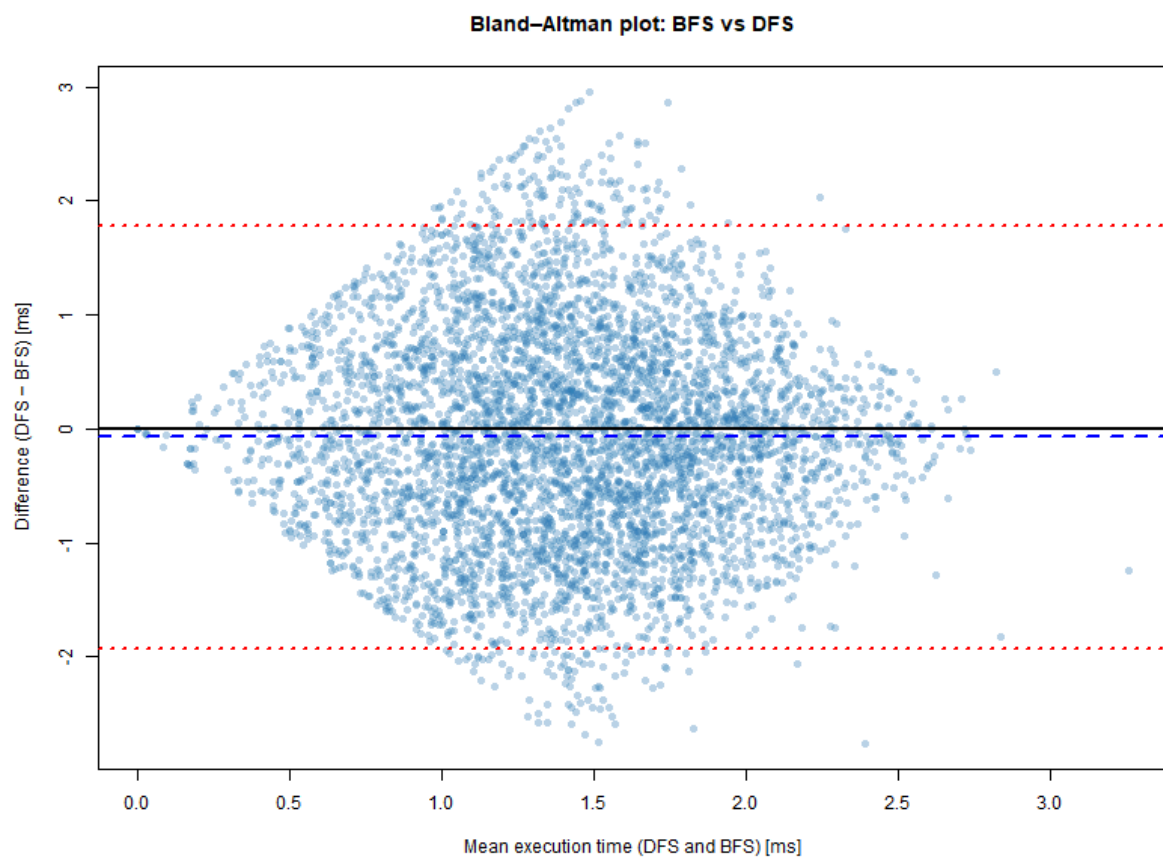
En primer lloc, es considera satisfeta la premissa d'independència de les observacions. Cada mesura de temps correspon a una execució independent dels algorismes sobre un graf determinat, amb parelles de nodes d'inici i final diferents. No existeix dependència temporal ni reutilització de resultats entre experiments, de manera que les observacions poden assumir-se com estadísticament independents.

Per tal de comparar adequadament els dos algorismes, s'ha introduït una nova variable anomenada diferència de temps, definida com:

$$Diff = T_{DFS} - T_{BFS}$$

Aquesta variable permet treballar amb dades aparellades, ja que cada valor de la diferència correspon a una mateixa instància del problema resolta amb BFS i DFS. Un valor negatiu indica que DFS ha estat més ràpid, mentre que un positiu indica un millor rendiment de BFS.

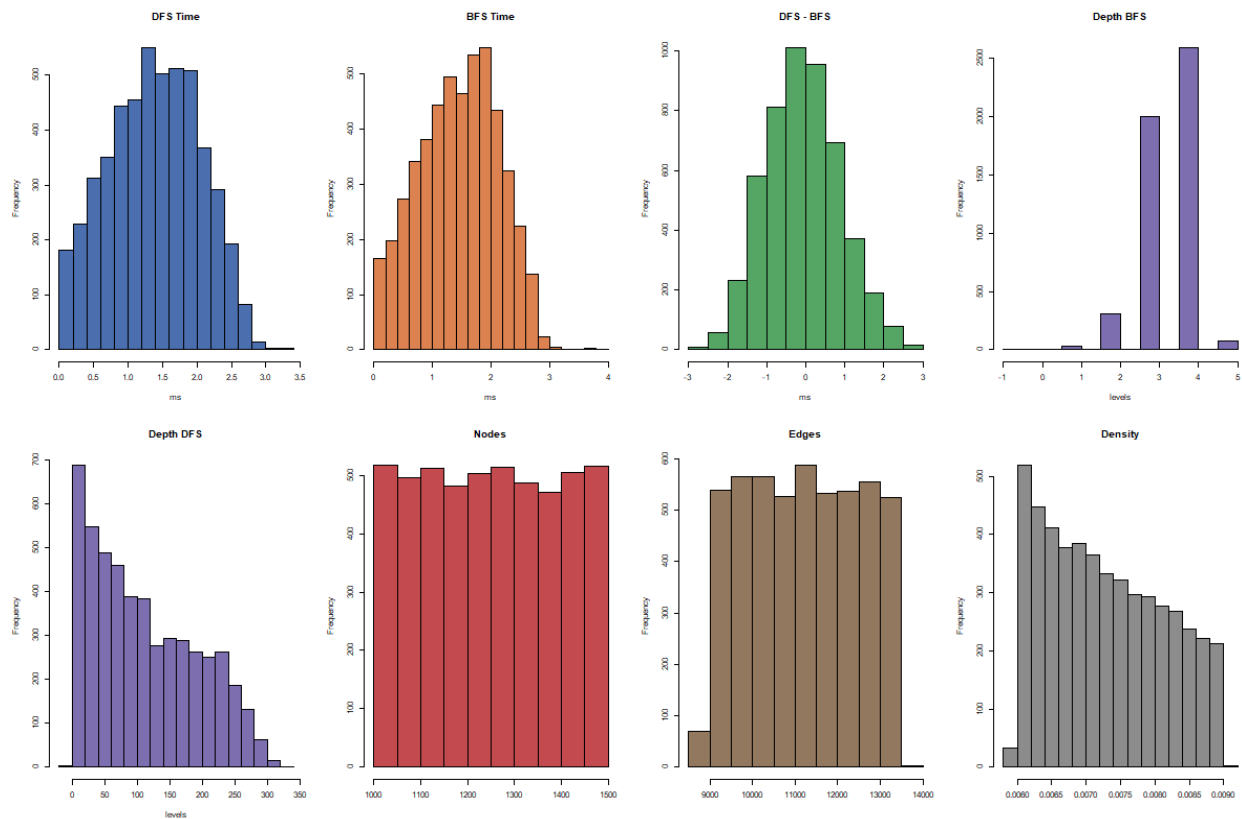
Pel que fa al tipus d'efecte, s'ha analitzat mitjançant un gràfic de Bland-Altman, que representa la diferència de temps en funció de la mitjana dels temps d'execució dels dos algorismes. L'anàlisi visual d'aquest gràfic mostra que la diferència es distribueix de manera relativament homogènia al voltant d'una mitjana constant, sense evidenciar una dependència clara respecte de la magnitud dels temps. Aquest comportament és indicatiu d'un efecte additiu, descartant un efecte multiplicatiu.



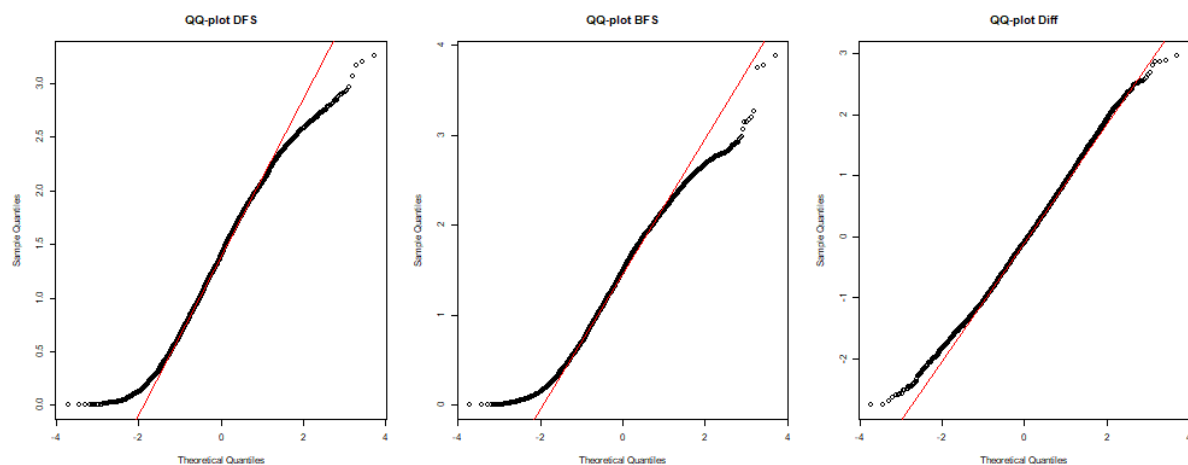
Respecte a la premissa de normalitat, s'ha analitzat la distribució de la variable Diff. Tant l'histograma com el QQ-plot de la diferència mostren una distribució aproximadament normal, amb una lleugera asimetria però sense desviacions greus. A més, cal tenir en compte que la mida de la mostra és elevada ($n = 5000$), fet que permet aplicar el teorema central del límit. En aquest context, encara que la normalitat no fos perfecta, els tests paramètrics com el test t aparellat continuen sent robustos i estadísticament vàlids.

Comparació de poblacions

Amb els histogrames següents veiem que de la diferència de temps d'execució segueix una distribució normal aproximadament.



L'anàlisi de la variable Diff mitjançant histogrames i gràfics de quantils també mostra que la seva distribució és aproximadament simètrica i compatible amb una distribució normal. Aquesta observació, com havíem dit abans, permet assumir el supòsit de normalitat necessari per a l'aplicació del T-Test.



Amb els gràfics de Q-Q Plot observem que els quantils de la diferència s'aproximen molt als quantils teòrics de la distribució normal, recolçant més encara la similitud.

Tot i que les distribucions dels temps d'execució de BFS i DFS mostren desviacions lleugeres respecte a la normalitat, l'anàlisi del diagrama Q-Q de la diferència entre ambdós temps indica una aproximació notable a una distribució normal. En conseqüència, es considera raonable assumir la normalitat de la diferència i aplicar una prova t per a dades aparellades.

Anàlisi descriptiu

Per comparar els temps d'execució de BFS i DFS s'ha aplicat un test t aparellat, ja que ambdós algorismes s'executen sobre els mateixos grafs i parelles de nodes.

Calculem els estadístics de la mitjana i la desviació, sense aplicar cap tipus de transformació, juntament amb l'estadístic de la diferència:

	Mitjana	SE
BFS	1.4531974	0.676791409043814
DFS	1.3796574	0.658721366583018
Diferència	-0.07354	0.947126654349816

Els resultats mostren una diferència mitjana de -0.0735 ms (DFS - BFS), amb un valor $t = -5.49$ i un p-valor inferior a 10^{-7} , la qual cosa permet rebutjar la hipòtesi d'igualtat de mitjanes. Utilitzant aquestes dades, som capaços d'estimar el càlcul d'un interval de confiança del 95% per a la mitjana de les diferències de temps, que és de $[-0.0998, -0.0473]$, completament negatiu, indicant que DFS és, de mitjana, més ràpid que BFS en les condicions experimentals analitzades.

Paired t-test

```
data: paired[[dfs_col]] and paired[[bfs_col]]
t = -5.4904, df = 4999, p-value = 4.209e-08
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 -0.09979889 -0.04728111
sample estimates:
mean difference
 -0.07354
```

S'ha ajustat un model de regressió lineal simple per analitzar la relació entre la diferència de temps d'execució entre DFS i BFS (variable resposta, definida com $DFS - BFS$) i la profunditat mitjana del recorregut (variable explicativa):

- El coeficient associat a la profunditat presenta un valor estimat de -0.0101601 , fet que indica que, a mesura que la profunditat augmenta en una unitat, la diferència de temps disminueix aproximadament en $0,01$ ms. Aquest resultat suggereix que DFS tendeix a ser progressivament més ràpid que BFS quan els recorreguts són més profunds.
- El valor del t-estadístic associat a aquest coeficient és $-33,91$, amb un p-valor inferior a $2 \cdot 10^{-16}$, la qual cosa proporciona una evidència estadística molt forta per rebutjar la hipòtesi nul·la d'absència de relació entre la profunditat i la diferència de temps.
- L'intercept del model és $0,509$, indicant que per a profunditats molt petites la diferència mitjana de temps és lleugerament positiva, és a dir, BFS pot ser marginalment més ràpid en recorreguts superficials.
- El coeficient de determinació R^2 és de $0,187$, cosa que implica que aproximadament el $18,7\%$ de la variabilitat observada en la diferència de temps d'execució queda explicada per la profunditat del recorregut. Tot i que una part important de la variabilitat continua sent atribuïble a altres factors no inclosos en el model, el test F global del model ($F = 1150$, $p < 2,2 \cdot 10^{-16}$) confirma que la regressió és globalment significativa.

```
Call:
lm(formula = Diff ~ Depth, data = paired)

Residuals:
    Min       1Q   Median       3Q      Max
-3.07083 -0.54642 -0.03022  0.56767  2.47221

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.5087111   0.0209908   24.23  <2e-16 ***
Depth       -0.0101601   0.0002996  -33.91  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.854 on 4998 degrees of freedom
Multiple R-squared:  0.1871, Adjusted R-squared:  0.1869
F-statistic: 1150 on 1 and 4998 DF, p-value: < 2.2e-16
```

CONCLUSIÓ

A través d'aquest estudi i després de la recollida i la posterior anàlisi de les dades, s'ha pogut validar amb precisió la hipòtesi proposada. Com a idea inicial, es preveia que possiblement la diferència de temps no era gaire significativa entre els dos algorismes amb enfocaments diferents sobre el problema. Tot i així, els resultats reflecteixen una altra situació.

Analitzant la variable Diff, resultat de calcular la diferència del temps del DFS i el BFS, es veu que tant la mitjana com l'interval de confiança de la distribució normal que segueix la variable es decanten per un resultat negatiu. Això deixa clar que el temps de BFS és més alt en general i, sobretot per profunditats més grans. D'altra banda, la variable Intercept és positiva, fet que mostra que de base, per a profunditats petites, aquest algorisme funciona millor.

Aquesta conclusió és lògica si es té en compte el comportament de cada procediment: el BFS acaba la seva execució un cop troba el primer camí i assegura que aquest és el més curt, mentre que el DFS troba camins possibles més ràpid, però ha d'explorar més opcions —que utilitzant una poda es redueixen bastant—. Relacionant l'estratègia de cerca amb els resultats, que per a profunditats molt curtes el BFS acaba ràpidament i la seva implementació és més simple, però en general el DFS a llarg termini ofereix una lleugera millora de temps per la seva gestió de memòria més eficient.

En definitiva, aquest estudi aporta evidència empírica sobre el comportament de dos algorismes de cerca en grafs molt emprats per diverses aplicacions. El resultat que afavoreix el temps d'execució del DFS enfront a l'altra opció, tot i a no ser radicalment determinant per escollir aquest algorisme enfront a l'altre, ajuda a considerar aspectes més tècnics de la gestió de memòria i la cerca de camins i obre la porta a futures anàlisis més detallades sobre aquest àmbit tan rellevant.

ANNEX

Les proves experimentals s'han dut a terme en un ordinador en un entorn amb les següents característiques:

- Sistema operatiu: Ubuntu 24.04.3 LTS (x86_64)
- Nucli (kernel): Linux 6.14.0-36-generic
- Paquets instal·lats: 1943 (dpkg), 19 (snap)
- Shell: Bash 5.2.21
- Processador (CPU): AMD Ryzen 7 5800X (16 nuclis lògics) @ 4.53 GHz
- Targeta gràfica (GPU): NVIDIA GeForce RTX 3060 Ti Lite
- Memòria RAM: 32 GB

```
.-/+00SSSS00+/-.  
' :+SSSSSSSSSSSSSSSSSS+: '  
  +SSSSSSSSSSSSSSSSSSyySSSS+-  
  .0SSSSSSSSSSSSSSSSSSdMMMMySSSS0.  
  /SSSSSSSSSSshdmmNmmNmmNmmHSSSSSS/  
  +SSSSSSSSshmydMMMMMMMMNddddySSSSSS+  
  /SSSSSSSSshNMMMyhhyyyyhmNMMMNhSSSSSSS/  
  .SSSSSSSSdMMMhSSSSSSSSshNMMMdSSSSSSS.  
  +SSSSshhyNMMMySSSSSSSSSSSyNMMMySSSSSS+  
  .SSyNMMMNyMMhSSSSSSSSSSShmmhSSSSSS0  
  .SSyNMMMNyMMhSSSSSSSSSSShmmhSSSSSS0  
  +SSSSshhyNMMMNySSSSSSSSSSSyNMMMySSSSSS+  
  .SSSSSSSSdMMMhSSSSSSSSShNMMMdSSSSSSS.  
  /SSSSSSSSshNMMMyhhyyyhdNMMMNhSSSSSSS/  
  +SSSSSSSSdmydMMMMMMMMdddySSSSSS+  
  /SSSSSSSSSSshdmmNmmNmmNmmNmmHSSSSSS/  
  .0SSSSSSSSSSSSSSSSSSdMMMMySSSS0.  
  +SSSSSSSSSSSSSSSSSSyyySSSS+-  
  ' :+SSSSSSSSSSSSSSSS+: '  
  .-/+00SSSS00+/-.
```

kate@kate

OS: Ubuntu 24.04.3 LTS x86_64
Kernel: 6.14.0-36-generic
Uptime: 3 mins
Packages: 1943 (dpkg), 19 (snap)
Shell: bash 5.2.21
Resolution: 2560x1440
DE: GNOME 46.0
WM: Mutter
WM Theme: Adwaita
Theme: Yaru-purple-dark [GTK2/3]
Icons: Yaru-purple [GTK2/3]
Terminal: gnome-terminal
CPU: AMD Ryzen 7 5800X (16) @ 4.853G
GPU: NVIDIA GeForce RTX 3060 Ti Lite
Memory: 1467MiB / 32005MiB



https://www.canva.com/design/DAG7wb4N4Mk/8sf7vi7fyaL3DLs9yFyXMQ/edit?utm_content=DAG7wb4N4Mk&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton